PICASSO: In-the-Loop Co-Exploration of Packages, D2D Interfaces, and Chiplets for AI Accelerator Nodes

Anonymous Authors

***Abstract*—Future AI/HPC nodes can hit the same target with one large die, a few chiplets, or many small ones, but the winner is no longer set by mapping and on-die architecture alone. Once traffic crosses die boundaries, package class, D2D latency and density, exposed edge, route budget, and memory attachment become first-class variables. We present PICASSO, an architecture-level framework that jointly searches mapping, microarchitecture, packages, D2D interfaces, chiplet organization, and memory while rejecting infeasible candidates in-loop. Across six workload motifs, projected baselines mis-rank winners in 8%–58% of design points, and even a strong repair-style baseline changes the winner in 14%–29% of communication-heavy regimes. PICASSO shows that package-oblivious search is reliable only at low traffic, interface efficiency often outranks package-only upgrades, and the broad middle of the design space usually favors 2–4 chiplets.**

***Index Terms*—chiplet accelerators, D2D interconnect, advanced packaging, design-space exploration, AI infrastructure, high-performance computing**

I. Introduction

Chiplets have become a first-order architectural mechanism for scaling AI infrastructure. Commercial systems such as TPUv4i, Cerebras, Tesla Dojo, Grace Hopper, MI300, and Ponte Vecchio make package organization, die-to-die (D2D) communication, and memory attachment visible at node scale rather than hiding them as backend implementation details [1–7]. At the same time, modern AI workloads increasingly amplify node-internal communication. Long-context prefills stream large activation volumes, decode exposes low-utilization and KV-heavy operating points, sparse MoE inference creates topology-sensitive bursts, and distributed training injects collective phases directly into the critical path [8–17].

This changes a design question that used to look mostly economic. A node target can often be realized as one large die, two or four compute chiplets, or a more fragmented multi-chiplet module. The decision is shaped by at least six tightly coupled variables: mapping, on-die architecture, chiplet organization, D2D interface, package class, and memory provisioning. Once a mapped workload pushes traffic across chiplet boundaries, it changes required PHY area, endpoint latency, link energy, exposed die edge, package route demand, and legal memory attachment. In other words, package and interface are not passive penalties appended after an accelerator is already chosen; they participate in determining which architectural neighborhoods are legal and worth exploring in the first place.

A large body of accelerator research has already shown that mapping and architecture must be explored together [18–29]. A parallel line of work made chiplet granularity, packaging cost, and heterogeneous D2D links architecturally visible [30–35]. The remaining gap is that package, D2D, and memory are still frequently externalized. Architecture-first flows search mapping and microarchitecture under a neutral envelope and repair package decisions later; partition-first flows optimize chiplet counts before checking route or memory legality; cost- or interface-driven projections improve the envelope without letting traffic patterns rewrite the design neighborhood. These decompositions are convenient, but they can prune the wrong winners before the missing variables are ever consulted.

This paper asks a node-ranking question with clear relevance to future AI/HPC systems: *when do package, interface, and memory terms change the identity of the best AI accelerator node?* We answer that question with PICASSO, a joint exploration framework over

$$d = (m, a, k, i, p, b), \tag{1}$$

where m is mapping, a is on-die architecture, k is chiplet organization, i is the D2D interface, p is the package class/topology, and b is memory and I/O provisioning. PICASSO does not claim tapeout-ready legality or global optimality. Its role is architecture-level ranking: keeping package, D2D, and memory in the inner loop early enough that they can invalidate or reprioritize architectural states before pruning occurs.

The main contributions are:

- We formulate chiplet AI node design as a *feasibility-coupled* exploration problem and show why projected search can prune the right design family before package/interface/memory decisions are ever consulted.
- We present PICASSO, which jointly explores mapping, architecture, partitioning, interfaces, packages, and memory with explicit models for PHY area, endpoint latency, route budget, memory attachment legality, idle-PHY energy, and yield-aware manufacturing cost.
- We introduce quantitative regime diagnostics together with a calibrated validation methodology, including anchor-based model grounding and a sampled open-stack backend-closure protocol for hard winner, boundary, and near-legality cases.
- We show across six workload motifs and strong projected baselines that in-loop package/interface/memory aware-

ness changes winner identity frequently enough to matter, especially in long-context inference, MoE routing, and collective-heavy regimes.

The rest of the paper proceeds as follows. Section II motivates projected-search failure and defines architecture-level feasibility. Section III presents the PICASSO framework and its node model. Section IV describes validation, workloads, traces, and baselines. Section V reports the main design rules. Sections VI and VII discuss limitations, reproducibility, and related work.

II. Motivation and Problem Formulation

The failure mode we care about is stronger than a scalar scoring error. Consider a projected search that explores only $\pi(d) = (m, a, k)$, ranks those states under a simplified objective, and decides package/interface/memory later. Once feasibility and communication pricing depend on (i, p, b), such a projection can eliminate all liftings of the ultimately superior joint design before the second stage even begins.

We rank a node with a normalized scalar objective,

$$J(d) = \hat{L}(d)^{\alpha} \hat{E}(d)^{\beta} \hat{C}(d)^{\gamma}, \tag{2}$$

where $\hat{L}$, $\hat{E}$, and $\hat{C}$ are normalized latency, energy, and cost, respectively. Unless stated otherwise, $\alpha = \beta = \gamma = 1$. We preserve the full Pareto frontier in the search, but Eq. (2) provides a consistent scalar ranking for winner-change analysis.

To characterize workload communication pressure we use the cross-die traffic ratio,

$$\phi = \frac{\text{bytes transferred across chiplets}}{\text{total bytes served by the node}}. \tag{3}$$

A low-ϕ node can often tolerate a projected flow because D2D penalties are occasional. A high-ϕ node repeatedly pays endpoint latency, serialization, energy/bit, and route demand; here, delaying package/interface/memory decisions is most dangerous.

In practice, projected flows fail in three distinct ways. *Communication-pricing errors* underprice repeated endpoint, serialization, and hop costs, which makes over-partitioned states look deceptively attractive. *Feasibility errors* arise when the same state later overruns exposed edge or package route budget once PHYs and memory channels are materialized. *Attachment errors* arise when the winner assumes a memory organization that is incompatible with the chosen chiplet/package configuration. Near regime boundaries, these three errors compound rather than cancel, which is why a seemingly small package or memory term can flip the winner.

Figure 1 illustrates the miss. The left half enumerates the architectural state that must be visible during search: mapped traffic, on-die resources, chiplet count, interface class, package class, and memory attachment. The right half shows a typical projected-search error. Under a neutral package and link envelope, an aggressively partitioned node can look favorable because smaller dies improve yield. Once the same node is evaluated under real D2D endpoint costs, perimeter limits, route budget, and memory-attachment legality, the apparent winner may become infeasible or simply less attractive than a 2–4-chiplet alternative. Late repair can only choose among descendants of the wrong projection.

This miss is structural rather than merely heuristic. If the outer search has already pruned the family containing the true joint winner, no second-stage repair can recover it; the repair stage can only optimize among the survivors of the first projection. Better repair heuristics may reduce the damage, but they cannot recover states that were never retained. This is why the comparison later in the paper uses strong projected baselines under a shared budget and seed policy rather than weak strawmen; the relevant question is whether delaying package/interface/memory awareness is acceptable at all.

This observation motivates the distinction we use throughout the paper. By *architecture-level feasibility* we mean three conditions: (i) the package and die organization must expose enough edge for the chosen PHYs and memory channels, (ii) the chosen package/topology must accommodate the aggregate route demand implied by the mapped traffic, and (iii) the memory attachment implied by b must be legal for the selected organization. This is intentionally weaker than signoff legality. We do *not* claim detailed package routing closure, signal-integrity closure, thermal closure, power-delivery closure, known-good-die (KGD) screening, or assembly-yield closure. The role of feasibility here is to prevent architectural search from wandering into obviously misleading neighborhoods.

The projected modes later used in the evaluation, together with the shared budget and logging policy that keep the comparison fair, are summarized in Table I. *Architecture-first* resembles mapping-plus-architecture co-search that repairs chiplet/package choices later. *Partition-first* optimizes chiplet granularity early but delays route and memory legality. *Cost/interface-first* begins with package envelopes and link technology but cannot let mapped traffic rewrite those choices. *Gemini-outer* is the strongest repair-style baseline: it performs a serious outer search over mapping and architecture under a neutral package/interface assumption and only later repairs chiplet, memory, and package decisions. If joint search still flips winners against this baseline, the issue is structural rather than heuristic.

III. The PICASSO Framework

Figure 2 shows the overall workflow. The proposal engine mutates one or two dimensions of the full state: mapping edits change fusion, tiling, placement, or temporal order; architecture edits resize arrays and buffers; partition edits split, merge, or rebalance chiplets; interface and package edits swap technology classes; memory edits change channel counts, attachment topology, or the use of an I/O die. The evaluator is not a passive scorer. It decomposes latency, energy, and cost, then rejects candidates that violate edge, route, or memory-attachment constraints.

A. Node model

PICASSO targets architecture-level ranking, not physical signoff. The compute side follows the mapping-aware accel-

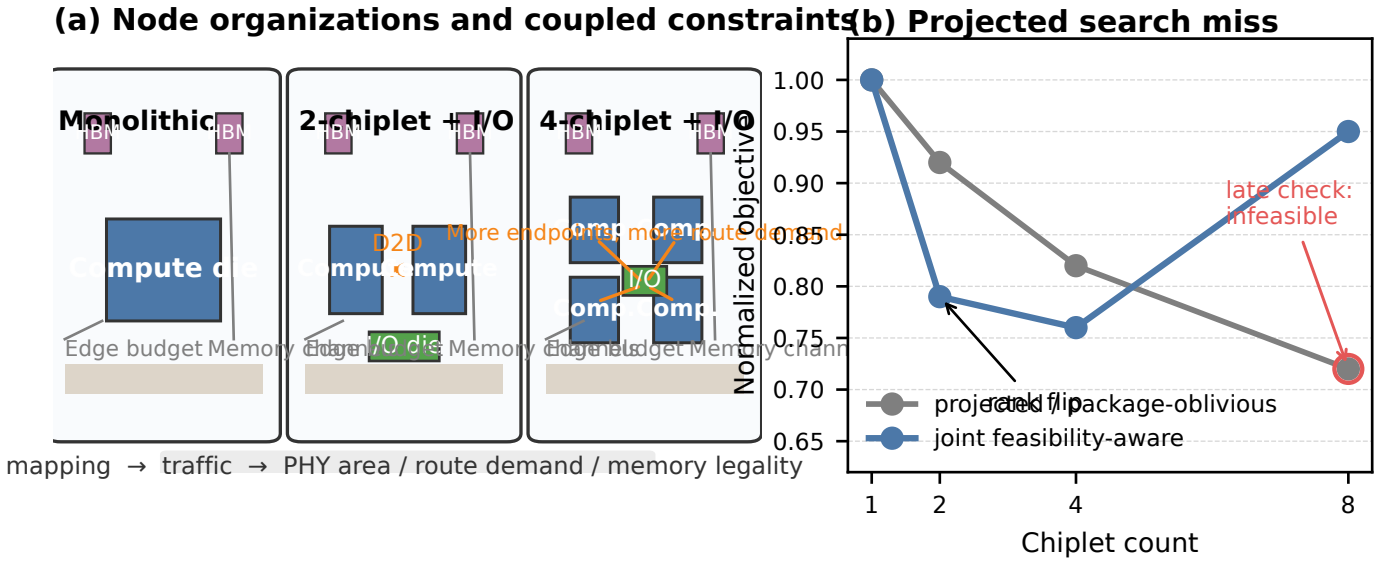

Fig. 1. **Why projected search misses the right neighborhood.** A future AI node can be organized as one large die, a few compute chiplets with direct-attached memory, or a more fragmented multi-chiplet module with an I/O die. Mapping-induced traffic determines PHY area, endpoint count, exposed edge, route demand, and legal memory attachment. A search that commits to (m, a, k) first can therefore select an aggressively partitioned candidate whose apparent advantage disappears once package, D2D, and memory are evaluated in-loop.

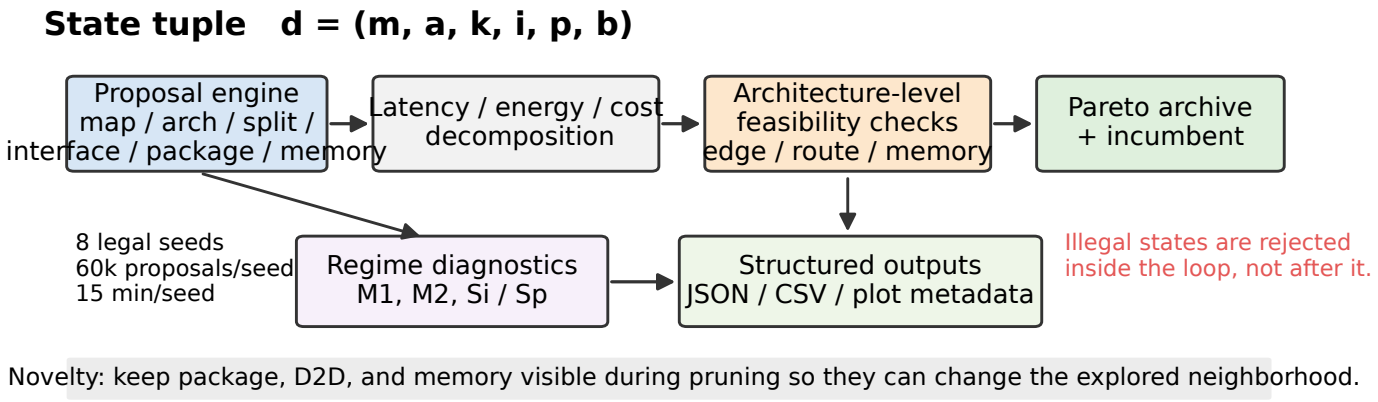

Fig. 2. **PICASSO keeps package, D2D, and memory in the inner loop.** Candidate states are proposed over the full tuple (m, a, k, i, p, b), evaluated by decomposed latency/energy/cost models, filtered by architecture-level feasibility checks, and tagged with regime diagnostics and structured logs. The search engine is deliberately simple; the novelty lies in the state representation and in-loop rejection of infeasible nodes.

erator tradition represented by Timeloop, Accelergy, MAESTRO, Tangram, Gemini, and related work [18–22, 36]. For a compute die u, we decompose area as

$$A_u = A_u^{\mathrm{cmp}} + A_u^{\mathrm{buf}} + A_u^{\mathrm{NoC}} + A_u^{\mathrm{ctrl}} + A_u^{\mathrm{PHY}}, \tag{4}$$

where the D2D term is

$$A_u^{\mathrm{PHY}} = \frac{B_u^{\mathrm{req}}}{\rho_i} + A_i^{\mathrm{adapt}}. \tag{5}$$

Here B_u^{req} is the aggregate link bandwidth demanded by the mapped traffic on die u, ρ_i is the bandwidth density of interface i, and A_i^{adapt} captures protocol and adaptation logic. This term explains why D2D cannot be bolted on later: traffic reshapes both die area and the edge resources that later enter feasibility.

Latency for a mapped layer or pipeline stage ℓ is modeled as

$$T_\ell = T_\ell^{\mathrm{cmp}} + T_\ell^{\mathrm{NoC}} + T_\ell^{\mathrm{NoP}} + T_\ell^{\mathrm{mem}}, \tag{6}$$

with package-level communication

$$T_\ell^{\mathrm{NoP}} = \sum_{e \in P_\ell} \left(\frac{V_{e,\ell}}{B_e(i,p)} + h_p^{\mathrm{hop}} + t_i^{\mathrm{ep}} \right), \tag{7}$$

where $V_{e,\ell}$ is the data volume on edge e during stage ℓ, $B_e(i,p)$ is the effective bandwidth achieved by edge e under interface i and package class p, h_p^{hop} is the topology-dependent per-hop delay contributed by the package, and t_i^{ep} is the fixed endpoint term of the chosen D2D interface. Equation (7) therefore prices both bulk transfer and the fixed per-transfer taxes that become visible in decode-like and bursty regimes.

Energy follows the same structure,

$$E_\ell = E_\ell^{\mathrm{cmp}} + E_\ell^{\mathrm{NoC}} + E_\ell^{\mathrm{NoP}} + E_\ell^{\mathrm{mem}} + E_\ell^{\mathrm{idlePHY}}. \quad (8)$$

The idle-PHY term is small in high-utilization dense regimes but material in decode-like or sparse-utilization settings. Manufacturing cost combines yield-adjusted die cost, package cost, and memory cost,

$$C(d) = \sum_{u \in U(d)} \frac{C_u^{\mathrm{raw}}}{Y_{\mathrm{die}}(A_u, t_u)} + \frac{C_{\mathrm{pkg}}(p, A_{\mathrm{pkg}}, N_{\mathrm{bond}})}{Y_{\mathrm{pkg}}(p)} + C_{\mathrm{mem}}(b), \quad (9)$$

where $U(d)$ is the set of dice in node design d, C_u^{raw} is the pre-yield manufacturing cost of die u, $Y_{\mathrm{die}}(A_u, t_u)$ is its die yield as a function of area A_u and process/defect parameter t_u, $C_{\mathrm{pkg}}(p, A_{\mathrm{pkg}}, N_{\mathrm{bond}})$ is the package cost under package class p for package footprint A_{pkg} and bond count N_{bond}, and $Y_{\mathrm{pkg}}(p)$ is the package yield term. Equation (9) makes chipletization explicitly non-monotonic: the first split may reduce yield-adjusted die cost, while later splits grow endpoint count, package complexity, and communication energy faster than they improve die economics.

B. Architecture-level feasibility

We model two coupled surrogate metrics. The first is edge demand,

$$G(d) = G_{\mathrm{mem}}(b) + G_{\mathrm{D2D}}(k, i), \quad (10)$$

where $G_{\mathrm{mem}}(b)$ is the perimeter consumed by the memory organization implied by b (for example, HBM stacks, direct-attached channels, or I/O-die-mediated channels) and $G_{\mathrm{D2D}}(k, i)$ is the perimeter consumed by the PHY macros required by chiplet organization k under interface i. The second is package route demand,

$$R(d) = \sum_{e \in P} \lambda_e h_e(d), \quad (11)$$

where λ_e is the traffic intensity placed on package edge e and $h_e(d)$ is the effective routing distance or hop demand induced by the topology chosen in d. A candidate is architecture-level feasible only if

$$G(d) \le G_{\mathrm{edge}}(p, k), \qquad R(d) \le R_{\mathrm{budget}}(p), \quad (12)$$

while the memory channels implied by b remain legal for the selected package/interface organization. Here $G_{\mathrm{edge}}(p, k)$ is the available exposed perimeter under package class p and organization k, and $R_{\mathrm{budget}}(p)$ is the routing richness afforded by the chosen package class. These surrogates do not replace floorplanning or signal integrity analysis; they keep perimeter, route richness, and memory attachment from disappearing during search.

C. Quantitative regime diagnostics

To interpret winner changes beyond a single scalar optimum, we define a *first-split margin*

$$M_1 = \Delta_1^{\mathrm{yield}} - (\Delta_1^{\mathrm{NoP}} + \Delta_1^{\mathrm{pkg}}), \quad (13)$$

and a *second-split margin*

$$M_2 = \Delta_2^{\mathrm{yield}} - (\Delta_2^{\mathrm{NoP}} + \Delta_2^{\mathrm{pkg}}). \quad (14)$$

The terms $\Delta_1^{\mathrm{yield}}$, Δ_1^{NoP}, and Δ_1^{pkg} measure, respectively, the normalized yield gain, NoP penalty, and package-side penalty associated with the *first* organizational split of a design family (typically monolithic $\rightarrow$ few chiplets at fixed node target). The second group of terms is defined analogously for the *next* split (few chiplets $\rightarrow$ many chiplets). Monolithic nodes dominate when $M_1 < 0$; a few chiplets dominate when $M_1 > 0$ but $M_2 < 0$; many-chiplet organizations require both margins to remain positive.

We also quantify whether it is more profitable to improve interface efficiency or to upgrade the package:

$$\begin{aligned} S_i &= \max_{i' \in I(d)} \left[\log J(d) - \log J(d \mid i \rightarrow i')\right], \\ S_p &= \max_{p' \in P(d)} \left[\log J(d) - \log J(d \mid p \rightarrow p')\right], \end{aligned} \quad (15)$$

where $I(d)$ and $P(d)$ denote the feasible interface and package upgrades available to design d while holding the other state variables fixed. The bracketed term is the reduction in log-objective achieved by a single upgrade. Thus S_i is the best multiplicative improvement obtainable by changing the interface alone, whereas S_p is the best improvement obtainable by changing only the package. When $S_i > S_p$, the dominant architectural lever is link efficiency rather than package sophistication alone.

D. Search algorithm and reproducible logs

The search engine is intentionally auditable. Each run begins from eight legal seeds spread across chiplet counts and package/interface classes. A move edits one or two dimensions of the state. Ten percent of proposals are coupled moves, such as split+interface or package+memory, to reduce pathological local minima. A candidate is accepted if it improves Eq. (2) or, for a non-improving move, with probability $\exp(-\Delta J/T_r)$ under a slowly decreasing temperature schedule. We run 60 k proposals per seed, cap wall-clock time at 15 min per seed, and keep a Pareto archive of at most 128 non-dominated points. The artifact exports per-round logs including proposal type, legality flags, rejection reason, current incumbent, current temperature, and cumulative rejection ratios.

IV. Validation and Experimental Methodology

A. Validation chain and claim boundaries

Because PICASSO is an architecture-level framework, the relevant question is not whether it reproduces every physical effect, but whether it preserves the ordering decisions that matter for architectural ranking. We therefore validate the framework along three orthogonal anchors before relying on the design rules.

First, the compute-side model is grounded against mapping-aware accelerator references in the style of Tangram, Gemini, and Timeloop [18–20]. We measure median rank-preserving error across 64 legal design pairs spanning four dense motifs, two process points, and eight legal seeds per point. The

TABLE I

PROJECTED SEARCH MODES AND SHARED SEARCH POLICY. THE UPPER BLOCK SUMMARIZES WHICH VARIABLES ARE VISIBLE BEFORE PRUNING; THE LOWER BLOCK LISTS THE COMMON BUDGET AND EMITTED FIELDS USED ACROSS ALL MODES.

Mode / item	State searched first	Typical blind spot or shared policy
Joint	(m, a, k, i, p, b)	Target flow
Stage-wise	(m, a, k, b) then (i, p)	Commits too early to the wrong chiplet neighborhood
Package-oblivious	(m, a, k) with fixed (i, p)	Underprices repeated D2D cost
Architecture-first	(m, a) then (k, i, p, b)	Good local core, wrong node organization
Partition-first	(a, k) then (m, i, p, b)	Split later fails edge/route/memory checks
Cost/interface-first	(k, i, p) then (m, a, b)	Optimizes the envelope, not the traffic that fills it
Gemini-outer repair	(m, a, k) under neutral (i, p, b), then repair	Feasibility arrives only after pruning
Seeds	8 legal seeds	Shared across all modes; spread across chiplet counts and interface/package classes
Budget	60k proposals/seed or 15 min/seed	Matched search effort for joint and projected flows
Move mix	map 0.25, arch 0.15, split 0.20, interface/package 0.15, memory 0.15, coupled 0.10	Same proposal families and evaluator across modes
Archive / logs	128 non-dominated points; round, seed, move family, accepted, legal, illegal reason, best score, design ID, elapsed time	Reproducibility fields emitted by every run

resulting error remains within a 10% envelope across the reference set. Second, the D2D layer is triangulated against public XSR-, USR-, and UCIe-like envelopes under a common normalization of bandwidth density, endpoint latency, and energy/bit [37–40]. Third, the package and yield model is sanity-checked against public OS, RDL, interposer, InFO, CoWoS, Foveros, and SoIC data, together with commercial system anchors whose organizations clearly inhabit the few-chiplet, memory-rich part of the design space [4, 5, 41–47].

Figure 3 summarizes these anchors. Importantly, they calibrate claim strength. We claim architecture-level ranking, not signoff legality. KGD screening, assembly yield, detailed package routing, signal integrity, thermal headroom, PDN, and reticle utilization are treated as boundary shifters rather than hidden signoff guarantees.

B. Sampled backend closure with the Combo-A open stack

To strengthen the paper beyond anchor-level validation, we add a sampled surrogate-to-backend closure protocol using an

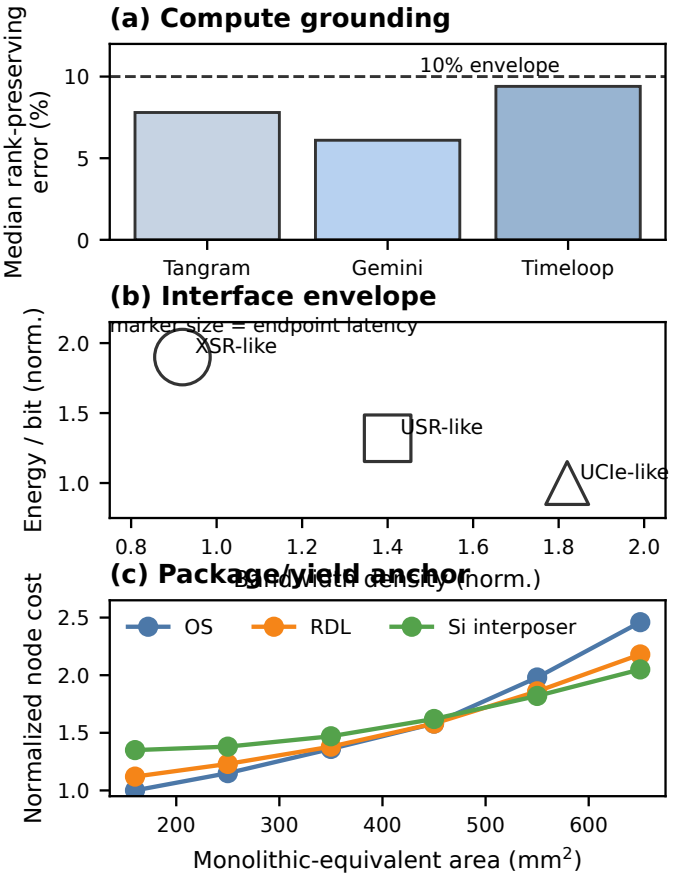

Fig. 3. **Validation chain.** PICASSO preserves accelerator ranking against mapping-aware references, maintains the expected public ordering of XSR/USR/UCIe-like interfaces, and reproduces the expected cost crossover trend of OS, RDL, and silicon-interposer packages as monolithic-equivalent area grows.

all-open stack. The replay flow uses OpenROAD/OpenLane-style floorplan and perimeter analysis for macro packing and edge pressure [48], a semi-physical package router for explicit route-capacity, hop, and escape-pressure replay, BookSim 2.0 for contention-sensitive NoP timing [49], and Ramulator 2.0 for attachment legality and queueing-sensitive memory timing [50]. Package cost itself is not re-inferred by this flow; it remains anchored by the industrial package cost data already used in the main model. The closure instead targets the higher-risk terms $G(d)$, $R(d)$, contention-sensitive NoP cost, and memory-side legality.

We do not replay the full design space. The artifact exports winner, boundary, near-legality, and disagreement cases from the full state tuple $d = (m, a, k, i, p, b)$, together with mapped traffic matrices, chiplet geometry, package/interface selections, and memory attachment choices. The replay stack then checks decision-preservation metrics rather than only raw-value error: top-1 agreement, legal precision/recall, and boundary drift. This protocol is intentionally concentrated on the hardest slices—especially the many-chiplet frontier—because route richness and memory attachment move that boundary more than they move the monolithic/few transition.

TABLE II
WORKLOAD MOTIFS, OPEN-SOURCE TRACE SOURCES, AND STRESS SIGNATURES. THE TABLE ALSO HIGHLIGHTS WHICH COMMUNICATION FEATURE EACH MOTIF IS MEANT TO EXPOSE.

Motif	Open-source trace source	Key setting / stress level	Why it matters
ResNet-50 streaming	Public ResNet-50 layer trace [55]	low ϕ, regular memory traffic	Corner where package-oblivious search can still look deceptively reliable
Long-context prefill	vLLM / SGLang serving trace [9, 51]	high ϕ, large activation movement	Sustained D2D movement stresses endpoint efficiency and route budget
KV-heavy decode	vLLM / SGLang decode trace [9, 51, 56]	low utilization, KV-heavy memory pressure	Exposes fixed endpoint cost and idle-PHY behavior
Dense decoder block	Public LLaMA-style block trace [57, 58]	med.–high ϕ, dense all-to-all stages	Captures dense LLM behavior without sparse routing
Mixtral-style MoE	Mixtral + open MoE routing instrumentation [11, 53, 54]	8 experts, top-2, 1.35× imbalance, 0.34 locality	Topology-sensitive bursts and many-chiplet feasibility
Megatron-style collective	Megatron-LM + ASTRA-sim/Chakra [12, 14, 52]	bucketized all-reduce/all-gather, 128 MB bucket	Collective pressure plus memory-attachment coupling

C. Workloads, trace sources, and technology sweeps

Table II consolidates the six workload motifs, their public trace sources, and the stress they inject into the design space. The serving-side prefill and decode motifs are derived from open-source LLM serving stacks, primarily vLLM and SGLang [9, 51]. The collective motif follows open Megatron-LM communication schedules and ASTRA-sim 2.0/Chakra-style trace generation [12–14, 52]. The MoE motif uses open Mixtral-style top-2 routing with locality and imbalance parameters informed by public MoE systems [11, 53, 54]. This is still weaker than proprietary production telemetry, but it is materially stronger than discussion-only proxy motifs because all traces now come from public, mainstream, open-source serving or training stacks.

We sweep node targets from 72 to 1024 TOPS, process assumptions from 12 nm to 7 nm, package classes across organic substrate (OS), redistribution-layer fan-out (RDL), and silicon interposer (SI), and interface classes across XSR-, USR-, and UCIe-like envelopes. Memory provisioning spans direct-attached DRAM channels, HBM-like stacks, and I/O-die-mediated attachment using LPDDR5-, GDDR6X-, and HBM-like cost/energy envelopes consistent with the updated Gemini codebase and public standards [59, 60]. All plots report normalized node-level values so trends can be compared across workloads and targets.

D. Baselines, ablations, and metrics

We compare joint search against the projected baselines from Table I. In addition, we report four targeted ablations: *interface-fixed*, *package-fixed*, *memory-off*, and *idle-PHY-off*. The explicit memory-off ablation is important because memory is not just an additive bandwidth scalar. Illegal memory attachment can make a design look attractive for the wrong reason.

The main metrics are: **winner-change rate**, which counts how often a weaker mode selects a different node than the joint flow; **reevaluated objective loss**, which measures the degradation incurred when the weaker mode's chosen node is rescored by the full evaluator; **boundary drift**, which measures how far the monolithic/few and few/many frontiers move under parameter sweeps; and **winner-stability tags**, which classify high-level claims as *robust*, *conditional*, or *boundary-sensitive* under weight and technology sweeps.

The strongest projected baseline is *Gemini-outer repair*. It first performs a strong mapping+architecture co-search inspired by Gemini-style outer loops, then repairs chiplet organization, package, interface, and memory. This baseline is intentionally competitive; if PICASSO still flips winners against it, the value of in-loop feasibility is difficult to dismiss as an artifact of weak decomposition.

V. RESULTS

A. Stronger projected baselines still miss winners

Figure 4 shows that the central message survives stronger projected baselines. Package-oblivious search is still the weakest mode, but the more realistic architecture-first, partition-first, and cost/interface-first flows also diverge sharply from joint search once communication becomes persistent. Winner-change rates climb from 8%–24% on the easiest corners to 31%–58% on MoE and collective traces. Even when the projected winner remains legal, it often inhabits the wrong regime: the full reevaluation loss for the chosen node reaches high single digits and is rarely a tie-level perturbation.

The strong repair-style baseline is the more important result. Across the prefill, dense-block, MoE, and collective slices, Gemini-outer repair still changes winner identity in 14%–29% of cases and incurs 2.8%–7.4% median reevaluated loss. This matters because it shows that the contribution is not "one more DSE heuristic." A serious mapping+architecture outer loop still misses the right neighborhood when package, D2D, and memory constraints are allowed to participate only after pruning.

B. Memory legality is a first-order architectural variable

The memory-off ablation confirms that memory is not simply an additive bandwidth knob. When legality checks are disabled, winner identity changes by 4% for CNN, 11% for prefill, 9% for decode, 13% for dense decoder blocks, 15% for the MoE trace, and 17% for the collective trace. The reevaluated losses range from 1.1% to 5.8%. Put differently, a meaningful fraction of apparently good designs were attractive only because the search was implicitly assuming illegal or unrealistic memory attachment.

This reframes why the 2–4-chiplet region is often strong. Its advantage is not driven by die economics alone. Moderate partitioning frequently creates a cleaner path to legal memory attachment than either a single very large die or a heavily

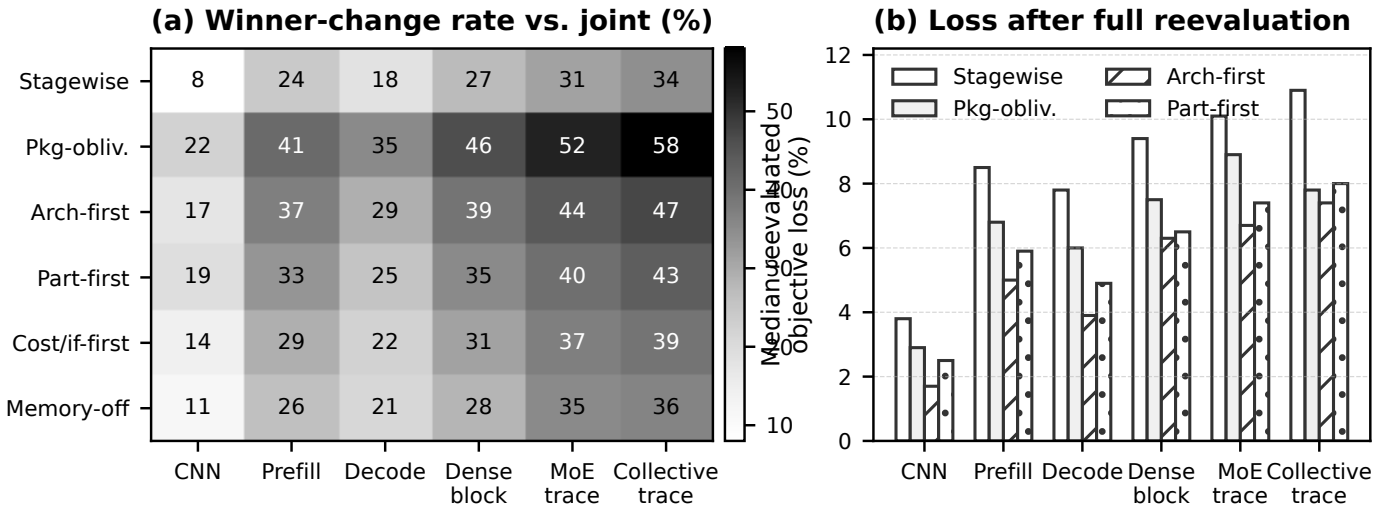

Fig. 4. **Stronger baselines still mis-rank chiplet nodes.** The heatmap reports winner-change rate relative to joint search; the companion bars report median reevaluated objective loss after the full model is applied. Communication-heavy traces magnify the gap rather than shrinking it.

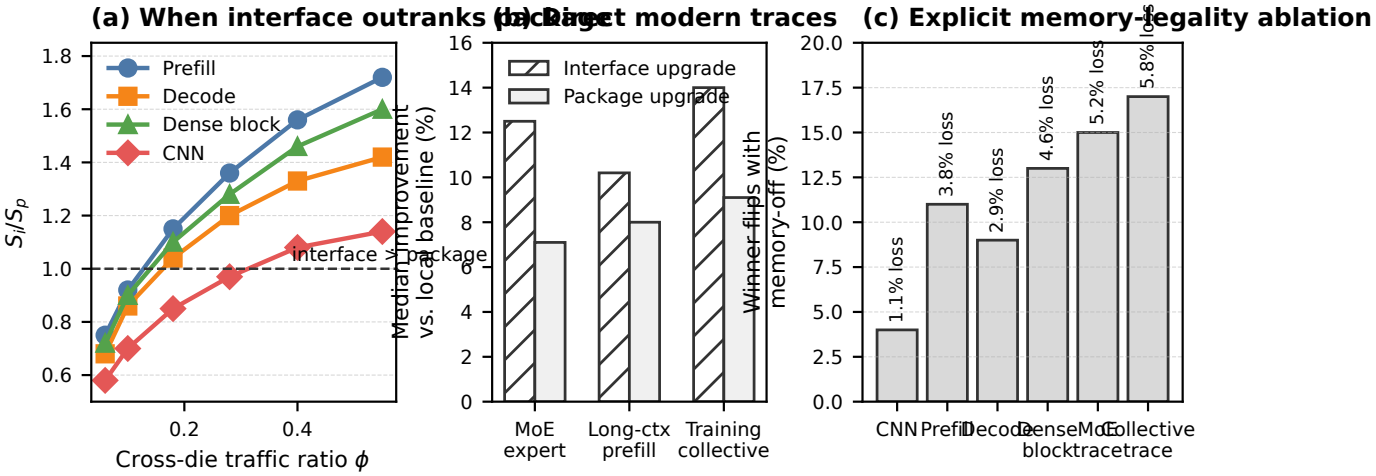

Fig. 5. **Interface importance and explicit memory ablation.** **(a)** The ratio S_i/S_p crosses one as sustained cross-die traffic grows, showing when link efficiency becomes the stronger architectural lever. **(b)** The trace-driven MoE and collective motifs show larger gains from interface upgrades than from package-only upgrades. **(c)** Disabling memory legality flips winners in precisely the high-pressure regimes where HBM-like attachment, channel count, and I/O-die organization compete with D2D edge usage.

fragmented organization with many competing PHYs and channels at the edge.

C. When interface outranks package

Figure 5(a) answers the first design question. At low ϕ, better packages help mainly by relaxing hop structure and route budget. Once cross-die movement becomes a repeated tax, however, interface quality dominates because the node repeatedly pays endpoint latency, serialization overhead, and energy/bit. For the prefill and dense-decoder regimes, S_i/S_p crosses one around $\phi \approx 0.18$–0.22 and continues to rise with traffic pressure. Decode crosses later, consistent with its lower aggregate traffic but higher sensitivity to fixed endpoint cost.

The direct trace-driven experiments strengthen the same point from a different angle. For the Mixtral-style MoE trace, an interface upgrade produces a larger median gain than a package-only upgrade because the dominant tax is repeated short-reach data movement rather than a single package bottleneck. The collective trace behaves similarly after route legality is satisfied: once the package is good enough, better link density and lower endpoint cost move the frontier more than another package step.

D. Why the few-chiplet middle is broad

Figure 6 explains the second design rule. The few-chiplet middle is not "magical"; it is the region in which the first split pays and the second does not. The first split usually buys substantial yield relief and memory-attachment flexibility. The second split pays a much steeper price in endpoint count, route demand, and package cost. Dense prefill-like processing shifts a node upward in ϕ and rightward in effective monolithic area A_{eq}, so the few-chiplet region arrives earlier. Decode

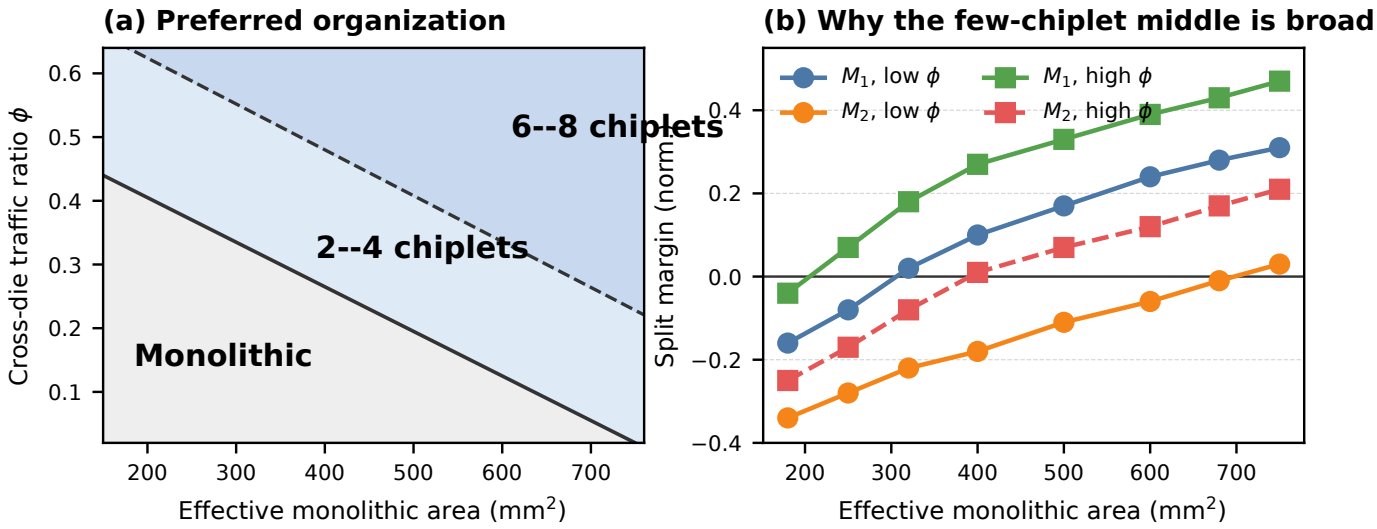

Fig. 6. **Preferred organization and split margins.** The monolithic/few boundary follows the sign of M_1; the few/many boundary follows the sign of M_2. The middle remains broad because the first split frequently buys yield and memory flexibility while the second rapidly increases endpoint count and package cost.

shifts less strongly in ϕ but increases the visibility of endpoint latency and idle-PHY effects, often changing the best interface without changing chiplet count.

The many-chiplet frontier is consistently the most fragile claim in the paper. MoE-like and collective-like motifs make 6–8 chiplets plausible only if route budget and memory attachment remain legal after the mapped traffic is imposed. Under a combined KGD+assembly-yield+thermal/PDN+reticle stress sweep, 58% of candidate 6–8-chiplet winners retreat to the 2–4-chiplet region, whereas the monolithic/few boundary moves by only 8%–13% under the same sweep.

E. Pareto structure, claim stability, and what actually kills projected winners

Figure 7 moves beyond the single scalar winner. The Pareto frontier shows that package class mostly changes the cost band: OS dominates the low-cost end, RDL occupies the middle, and silicon interposers mainly appear in the latency-aggressive corner. Interface upgrades more often push the frontier outward, whereas package-only upgrades can move a node primarily along the cost axis. The energy decomposition helps explain why joint search pays off: compute energy varies comparatively little across modes, while NoP and idle-PHY energy widen the gap. Over-partitioned nodes waste energy on communication inefficiency before they fail outright.

Figure 8 is the claim-boundary view of the results. Three conclusions remain robust across the explored public envelopes: (i) package-oblivious search becomes unreliable quickly once communication is sustained, (ii) interface improvements become the dominant lever before package sophistication alone does in high-ϕ regimes, and (iii) the few-chiplet middle dominates a broad swath of the design space whenever $M_1 > 0$ and $M_2 < 0$. The exact monolithic/few boundary is conditional on defect density and package cost. The exact few/many boundary is boundary-sensitive because it depends strongly on route richness, endpoint efficiency, and the objective's tolerance for cost.

TABLE III
ILLEGAL BREAKDOWN AMONG THE TOP 64 PROJECTED CANDIDATES UNDER FULL REEVALUATION. EACH CELL REPORTS EDGE / ROUTE / MEMORY PERCENTAGES. STRONG REPAIR-STYLE BASELINES REMAIN MEMORY- AND ROUTE-SENSITIVE ONCE MODERN TRAFFIC MOTIFS ARE IMPOSED.

Workload	Arch-first	Part-first	Gemini-outer
Prefill	12 / 31 / 57	9 / 29 / 62	11 / 24 / 65
Decode	5 / 18 / 77	4 / 16 / 80	5 / 13 / 82
Dense block	8 / 27 / 65	7 / 24 / 69	8 / 21 / 71
MoE trace	10 / 41 / 49	8 / 46 / 46	11 / 34 / 55
Collective trace	21 / 23 / 56	18 / 31 / 51	20 / 18 / 62

This is also where the sampled Combo-A backend-closure protocol of Section IV-B is most informative. Because the replay stack is concentrated on winner, boundary, and near-legality cases rather than on easy corners, it primarily sharpens claim strength instead of replacing the surrogate. In practice, the replay is most consequential on the many-chiplet frontier, which is exactly why Figure 8(b) labels many-chiplet viability as boundary-sensitive across several workloads.

Table III makes the projected-search failure mode more diagnostic. The top 64 candidates emitted by each strong projected baseline were reevaluated under the full joint model and classified by the first constraint that invalidated them: edge shortage, route overuse, or illegal memory attachment. Dense decode-like cases are predominantly memory-illegal; the projected flow often assumes more channels, more convenient HBM placement, or a cheaper I/O organization than the selected node can actually support. MoE and collective traces accumulate both route and memory violations because their burstiness and collective fan-out stress the package fabric and the chip edge simultaneously.

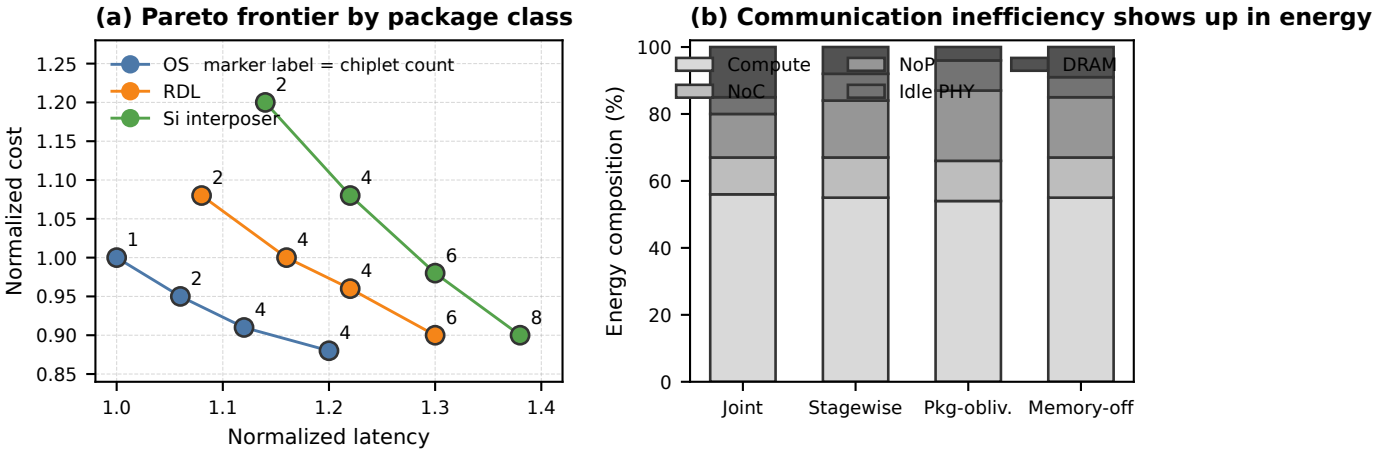

Fig. 7. **Pareto structure and energy decomposition.** Advanced packages do not uniformly dominate. OS-based nodes occupy the low-cost band, RDL-based nodes populate the middle, and interposer-based nodes appear mainly in the aggressive low-latency corner. Most of the energy gap between joint and weaker modes comes from NoP inefficiency and idle-PHY overhead rather than from compute energy.

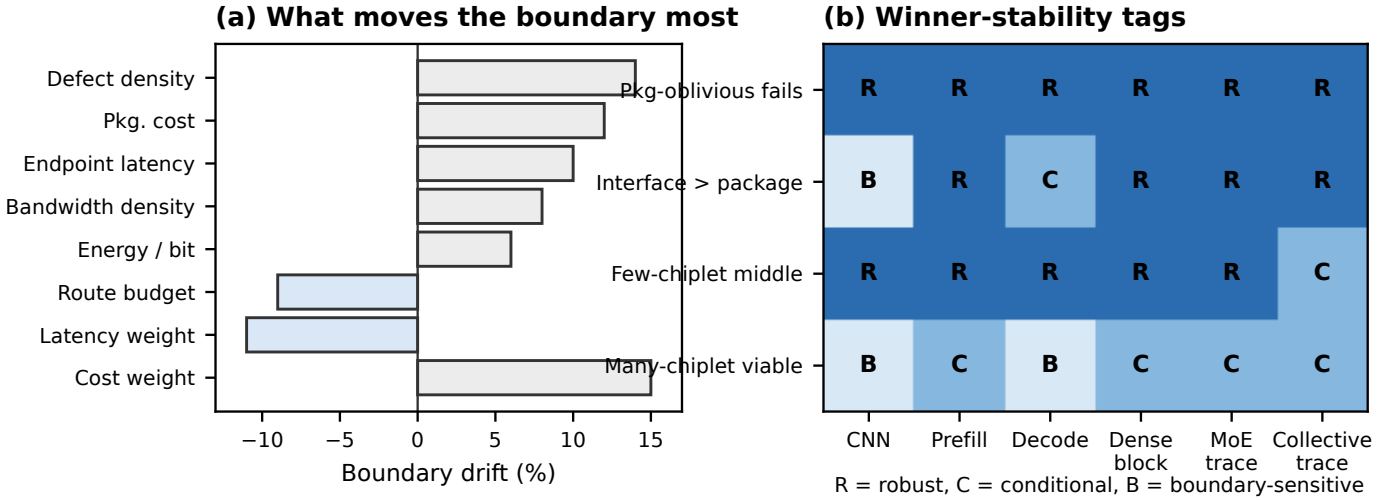

Fig. 8. **Sensitivity and calibrated claim strength.** Boundary drift identifies which parameters move the regime boundaries most; the stability heatmap distinguishes robust, conditional, and boundary-sensitive conclusions. The paper deliberately reports calibrated claim strength instead of universal laws.

F. Objective emphasis and practitioner guidance

The regime picture is stable to moderate changes in objective emphasis, but not all boundaries move equally. The left block of Table IV summarizes the first-order interpretation of the decision boundary in Eq. (2): emphasizing latency expands the region where low endpoint latency and route-richer packages become worthwhile earlier; emphasizing energy penalizes over-partitioned decode-like nodes because idle-PHY energy and repeated endpoint taxes become more visible; and emphasizing cost broadens the OS/RDL and 2–4-chiplet regime while suppressing many-chiplet solutions unless M_2 remains strongly positive.

The right block of Table IV condenses the same picture into a regime-oriented playbook. The point is not that every node should be forced into one template, but that workload characterization should tell an architect which lever to inspect first. This is especially relevant for SC-style system studies, where node architecture, serving policy, and cluster communication are often optimized by different teams. A regime-aware node model prevents those efforts from pulling in contradictory directions.

VI. Discussion, Limitations, and Reproducibility

The most portable outcome of this study is not a single universally optimal node organization. It is a compact set of state variables and diagnostics that travel across workloads: the effective monolithic area A_{eq}, the cross-die traffic ratio ϕ, the split margins M_1 and M_2, and the sensitivity ratio S_i/S_p. These quantities explain *why* a winner changes, which is more useful to an architect than a bare list of "best points." A second lesson is that PICASSO composes naturally with prior flows rather than replacing them wholesale: a Tangram/Gemini-style mapper can still provide a powerful inner search over mapping and architecture, while package, D2D, and memory are allowed to invalidate or reprioritize states before pruning [18, 19, 31, 32].

TABLE IV

PRACTICAL INTERPRETATION OF REGIME SHIFTS. THE LEFT BLOCK SUMMARIZES HOW OBJECTIVE WEIGHTS MOVE THE BOUNDARY; THE RIGHT BLOCK CONDENSES THE REGIME-ORIENTED PLAYBOOK USED TO READ THE PHASE MAP.

Weight	First-order architectural effect
Latency α	Expands the region where low endpoint latency and route-richer packages become worthwhile earlier.
Energy β	Penalizes over-partitioned decode-like nodes because idle-PHY and repeated endpoint costs become more visible.
Cost γ	Broadens the OS/RDL and 2–4-chiplet regime and suppresses many-chiplet solutions unless M_2 remains strongly positive.

Regime	Telltale signals	Likely organization	First lever to inspect
Small dense CNN / modest node	Low ϕ, modest A_{eq}, regular memory traffic	Monolithic or 2 chiplets	Mapping/on-die dataflow before package changes
Long-context prefill / dense blocks	Sustained cross-die activation movement, large A_{eq}	2–4 chiplets	Interface density and endpoint latency before package-only upgrades
KV-heavy decode	Moderate traffic but low utilization and latency sensitivity	2–4 chiplets	Fixed endpoint cost, idle-PHY behavior, and memory legality
MoE / collective-heavy	Bursty or fan-out-heavy traffic, route pressure, memory stress	2–4 by default; 6–8 only with strong envelopes	Route-rich package plus in-loop legality checking

The limitations are equally important. PICASSO does not claim detailed package routing, signal integrity, cycle-accurate DRAM timing, thermal simulation, power-delivery closure, KGD test cost, assembly yield, spare strategy, or detailed binning policies inside the surrogate itself. Some of these factors are not merely backend details; they can move the boundary between a few chiplets and many chiplets. That is why the paper labels conclusions as robust, conditional, or boundary-sensitive rather than presenting a universal law. The sampled Combo-A closure of Section IV-B is meant to stress exactly these boundary cases without overstating signoff fidelity.

Reproducibility is treated as an artifact concern rather than as a long prose section in the paper. The mandatory SC26 AD appendix and accompanying artifact README provide the concrete setup steps, sampled backend-closure workflow, figure-rebuild commands, and environment details needed to reproduce the main claims [61–63].

VII. RELATED WORK

Mapping-aware accelerator modeling. Prior accelerator work established that DNN performance cannot be evaluated by on-die resources alone; mapping, data movement, and memory hierarchy are inseparable from microarchitecture [20–29]. Tangram, SET, and Gemini sharpened this lesson by co-exploring mapping and architecture directly [18, 19, 36]. PICASSO extends that line by arguing that package, D2D, and memory must also be visible before pruning.

Chiplet AI accelerators and modular nodes. Simba demonstrated the importance of multi-chip inference organization, NN-Baton studied chiplet granularity, Chiplet Actuary quantified cost, and heterogeneous-D2D work showed that interface diversity can be architecturally meaningful [30–33]. More recent work expands the space further with heterogeneous multi-model scheduling, hybrid-bonding-based low-batch LLM inference, wafer-scale LLM co-exploration, wafer-scale fabrics for training, and fine-grained communication/memory co-design in MCM accelerators [34, 35, 64–66]. PICASSO differs by centering the *winner-identity* question and by explicitly studying when projected search fails.

Packages, interfaces, and system-scale communication. Public D2D and packaging work provides enough information to construct ranking-oriented envelopes without claiming signoff fidelity, including UCIe, XSR/USR-style short-reach links, EMIB, fan-out/RDL, CoWoS, Foveros, and SoIC [37–42, 44–47, 67]. Classical and modern interconnect work remains relevant because hop structure and route demand still shape architectural choices [68–73]. On the workload side, LLM serving and training systems continue to expose prefill/decode asymmetry, collective-critical behavior, and communication-heavy execution paths [8, 9, 12–14, 16, 17, 51, 56, 74–83]. PICASSO's contribution is to expose package/interface/memory early enough that the search neighborhood itself changes.

VIII. CONCLUSION

PICASSO shows that future AI/HPC nodes cannot be ranked reliably by mapping and on-die architecture alone once traffic crosses die boundaries. Keeping package, D2D, and memory in the inner loop changes which designs remain legal, which candidates survive pruning, and ultimately which node wins. Across the explored workloads and technology envelopes, the design space resolves into three interpretable regimes: monolithic when both area pressure and communication are modest, a broad 2–4-chiplet middle when the first split pays and the second usually does not, and many-chiplet organizations only when route richness, endpoint efficiency, and memory attachment remain favorable together. For future chiplet AI nodes, package/interface/memory choices are therefore architectural variables, not backend afterthoughts.

REFERENCES

[1] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter C. Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David A. Patterson. "Ten Lessons from Three Generations Shaped Google's TPUv4i". In: *Proceedings of ISCA*. 2021.

[2] Joules Vasiljevic, Luka Bajic, Dragan Capalija, Stefan Sokorac, Dusan Ignjatovic, Lazar Bajic, Milos Trajkovic, Ivan Hamer, Igor Matosevic, Aleksandar Cejkov, Utku Aydonat, Tianlin Zhou, Syed Zaigham Gilani, Arthur Paiva, Jason Chu, Dusan Maksimovic, Shawn A. Chin, Ziad Moudallal, Ali Rakhmati, Sanjeev Nijjar, Ajit Bhullar, Boris Drazic, Calvin Lee, Jerry Sun, Kevin Kwong, John Connolly, Matthew Dooley, Hamad Farooq, Jennifer Y. T. Chen, Mike Walker, Karan Dabiri, Kathy Mabee, Radmilo S. Lal, Nayana Rajatheva, Rajeev Retnamma, Slaven Karodi, David Rosen, Emiliano Munoz, Andrew Lewycky, and Aleksandar Knezevic. "Compute Substrate for Software 2.0". In: *IEEE Micro* 41.2 (2021).

[3] Ganesh Venkataramanan Talpes, Daniel Williams, and Dhiraj D. Sarma. "Dojo: The Microarchitecture of Tesla's Exa-Scale Computer". In: *Proceedings of Hot Chips*. 2022.

[4] Jonathon Evans, Michael Andersch, Vikram Sethi, Gonzalo Brito, and Vishal Mehta. *NVIDIA Grace Hopper Superchip Architecture In-Depth*. NVIDIA Technical Blog. 2022.

[5] Advanced Micro Devices. *AMD Instinct MI300 Series Architecture White Paper*. White paper. 2024.

[6] Intel Corporation. *Ponte Vecchio: An Exascale GPU for HPC and AI*. Hot Chips presentation / product overview. 2022.

[7] NVIDIA Corporation. *NVIDIA Blackwell Platform Architecture*. Architecture white paper. 2024.

[8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Hao Zhang, Binhang Yuan, Junxian He, Shantanu Mittal, and Ion Stoica. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 2023.

[10] Noam Shazeer. "Fast Transformer Decoding: One Write-Head is All You Need". In: *arXiv preprint arXiv:1911.02150* (2019).

[11] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. "Mixtral of Experts". In: *arXiv preprint arXiv:2401.04088* (2024).

[12] Mohammad Shoeybi, Mostafa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism". In: *arXiv preprint arXiv:1909.08053* (2019).

[13] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. "DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters". In: *Proceedings of KDD*. 2020.

[14] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. "ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-Model Training at Scale". In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2023.

[15] Sylvain Jeaugey. *NCCL 2.4 and Scalable Collective Communication for Deep Learning*. NVIDIA Developer Blog. 2019.

[16] Kishore Punniyamurthy, Khaled Hamidouche, and Bradford Beckmann. "Optimizing Distributed ML Communication with Fused Computation-Collective Operations". In: *Proceedings of SC*. 2024.

[17] Jiajun Huang, Sheng Di, Xiaodong Yu, Yujia Zhai, Jinyang Liu, Zizhe Jian, Xin Liang, Kai Zhao, Xiaoyi Lu, Zizhong Chen, Franck Cappello, Yanfei Guo, and Rajeev Thakur. "hZCCL: Accelerating Collective Communication with Co-Designed Homomorphic Compression". In: *Proceedings of SC*. 2024.

[18] Jingwei Cai, Zuotong Wu, Sen Peng, Yuchen Wei, Zhanhong Tan, Guiming Shi, Mingyu Gao, and Kaisheng Ma. "Gemini: Mapping and Architecture Co-exploration for Large-scale DNN Chiplet Accelerators". In: *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 2024.

[19] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. "Tangram: Optimized Coarse-Grained Dataflow for Scalable Neural-Network Accelerators". In: *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2019.

[20] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel S. Emer. "Timeloop: A Systematic Approach to DNN Accelerator Evaluation". In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2019.

[21] Yannan N. Wu, Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yiran Chen, and Joel S. Emer. "Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs". In: *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 2019.

[22] Hyoukjun Kwon, Michael Pellauer, and Tushar Krishna. "MAESTRO: An Open-Source Infrastructure for Mod-

eling Dataflows within Deep Learning Accelerators". In: *IEEE Micro* 40.3 (2020).

[23] Ananda Samajdar, Jan Moritz Joseph, Steven Hughes, Arun Pattnaik, and Tushar Krishna. "SCALE-Sim: Systolic CNN Accelerator Simulator". In: *arXiv preprint arXiv:1811.02883* (2018).

[24] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks". In: *IEEE Journal of Solid-State Circuits* 52.1 (2017).

[25] Yu-Hsin Chen, Tien-Ju Yang, Joel S. Emer, and Vivienne Sze. "Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices". In: *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9.2 (2019).

[26] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. "MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects". In: *Proceedings of ASPLOS*. 2018.

[27] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel S. Emer. "SCNN: An Accelerator for Compressed-Sparse Convolutional Neural Networks". In: *Proceedings of the International Symposium on Computer Architecture (ISCA)*. 2017.

[28] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. "TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory". In: *Proceedings of ASPLOS*. 2017.

[29] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. "DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning". In: *Proceedings of ASPLOS*. 2014.

[30] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Stephen G. Tell, Yanqing Zhang, William J. Dally, Joel S. Emer, C. Thomas Gray, Brucek Khailany, and Stephen W. Keckler. "Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture". In: *Proceedings of the IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 2019.

[31] Zhanhong Tan, Haoran Cai, Ruobing Dong, and Kaisheng Ma. "NN-Baton: DNN Workload Orchestration and Chiplet Granularity Exploration for Multichip Accelerators". In: *Proceedings of ISCA*. 2021.

[32] Yutong Feng and Kaisheng Ma. "Chiplet Actuary: A Quantitative Cost Model and Multi-Chiplet Architecture Exploration". In: *Proceedings of the Design Automation Conference (DAC)*. 2022.

[33] Yutong Feng, Dongxuan Xiang, and Kaisheng Ma. "Heterogeneous Die-to-Die Interfaces: Enabling More Flexible Chiplet Interconnection Systems". In: *Proceedings of MICRO*. 2023.

[34] Mohanad Odema, Luke Chen, Hyoukjun Kwon, and Mohammad Abdullah Al Faruque. "SCAR: Scheduling Multi-Model AI Workloads on Heterogeneous Multi-Chiplet Module Accelerators". In: *Proceedings of MICRO*. 2024.

[35] Taishu Sheng, Guangyu Sun, and Dezun Dong. "COMET: Communication and Memory Co-Design for Fine-Grained AI Inference in MCM Accelerators". In: *Proceedings of HPCA*. 2026.

[36] Jingwei Cai, Yuchen Wei, Zuotong Wu, Sen Peng, and Kaisheng Ma. "Inter-layer Scheduling Space Definition and Exploration for Tiled Accelerators". In: *Proceedings of the International Symposium on Computer Architecture (ISCA)*. 2023.

[37] Universal Chiplet Interconnect Express Consortium. *Universal Chiplet Interconnect Express (UCIe) Specification Revision 2.0*. 2024.

[38] Debendra Das Sharma, Gholamreza Pasdast, Zhenxiao Qian, and Kerem Aygun. "UCIe: An Open Industry Standard for Innovations with Chiplets at Package Level". In: *IEEE Transactions on Components, Packaging and Manufacturing Technology* 12.9 (2022).

[39] Behzad Dehlaghi, Neil Wary, and Tony C. Carusone. "Ultra-Short-Reach Interconnects for Die-to-Die Links: Global Bandwidth Demands in Microcosm". In: *IEEE Solid-State Circuits Magazine* 11.2 (2019).

[40] Ravi Shivnaraine, Marcus Van Ierssel, Kamran Farzan, Domenic Diclemente, George Ng, Nan Wang, Javid Musayev, G. Dutta, M. Shibata, A. Moradi, H. Vahedi, M. Farzad, P. Kainth, M. Yu, N. Nguyen, J. Pham, and Allan McLaren. "A 26.5625-to-106.25 Gb/s XSR SerDes with 1.55 pJ/b Efficiency in 7 nm CMOS". In: *Proceedings of ISSCC*. 2021.

[41] Ravi Mahajan, Robert Sankman, Neha Patel, Dae-Woo Kim, Kerem Aygun, Zhiguo Qian, Yidnekachew Mekonnen, Islam Salama, Sujit Sharan, Deepti Iyengar, and Debendra Mallik. "Embedded Multi-Die Interconnect Bridge (EMIB): A High Density, High Bandwidth Packaging Interconnect". In: *Proceedings of the Electronic Components and Technology Conference (ECTC)*. 2016.

[42] John H. Lau, C. P. Wong, Ning-Cheng Lee, and S. W. Ricky Lee. "Redistribution Layers (RDLs) for 2.5D/3D IC Integration". In: *Journal of Microelectronics and Electronic Packaging* 11.1 (2014).

[43] Tzu-Ming Wu, Yasumitsu Tsukada, and Wei Chen. "Materials and Mechanics Issues in Flip-Chip Organic Packaging". In: *Proceedings of ECTC*. 1996.

[44] Tao Li, Jingxin Hou, Jun Yan, Rong Liu, Haoyang Yang, and Zhiyuan Sun. "Chiplet Heterogeneous Integration Technology: Status and Challenges". In: *Electronics* 9.4 (2020).

[45] Taiwan Semiconductor Manufacturing Company. *CoWoS Technology Overview*. White paper. 2024.

[46] Intel Corporation. *Foveros and Foveros Direct 3D Packaging Technology Overview*. White paper. 2023.

[47] Taiwan Semiconductor Manufacturing Company. *TSMC SoIC Technology and 3D Chip Stacking Overview*. White paper. 2024.

[48] Mohamed Shalan and Tim Edwards. "Building OpenLANE: A 130nm OpenROAD-Based Tapeout-Proven Flow". In: *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 2020.

[49] Nan Jiang, James Balfour, Daniel U. Becker, Brian Towles, William J. Dally, George Michelogiannakis, and John Kim. "A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator". In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2013, pp. 86–96.

[50] Haocong Luo, Yahya Can Tugrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, and Onur Mutlu. "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator". In: *IEEE Computer Architecture Letters* (2024), pp. 1–4.

[51] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. "SGLang: Efficient Execution of Structured Language Model Programs". In: *arXiv preprint arXiv:2312.07104* (2024).

[52] Srinivas Sridharan, Taekyung Heo, Louis Feng, Zhaodong Wang, Matt Bergeron, Wenyin Fu, Shengbao Zheng, Brian Coutinho, Saeed Rashidi, Changhai Man, and Tushar Krishna. "Chakra: Advancing Performance Benchmarking and Co-Design Using Standardized Execution Traces". In: *arXiv preprint arXiv:2305.14516* (2023).

[53] William Fedus, Barret Zoph, and Noam Shazeer. "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23 (2022).

[54] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2021.

[55] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep Residual Learning for Image Recognition". In: *Proceedings of CVPR*. 2016.

[56] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. "DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving". In: *arXiv preprint arXiv:2401.09670* (2024).

[57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023).

[58] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288* (2023).

[59] JEDEC Solid State Technology Association. *HBM3 DRAM Standard JESD238A*. 2023.

[60] Hybrid Memory Cube Consortium. *Hybrid Memory Cube Specification Version 2.1*. 2015.

[61] SC26 Conference. *SC26 Papers Submission Guidelines*. Conference website. 2026.

[62] SC26 Conference. *SC26 Reproducibility Initiative*. Conference website. 2026.

[63] SC26 Conference. *SC26 AD/AE Appendices*. Conference website. 2026.

[64] Cong Li, Yihan Yin, Xintong Wu, Jingchen Zhu, Zhutianya Gao, Dimin Niu, Qiang Wu, Xin Si, Yuan Xie, Chen Zhang, and Guangyu Sun. "2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference". In: *Proceedings of ISCA*. 2025.

[65] Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. "WSC-LLM: Efficient LLM Service and Architecture Co-Exploration for Wafer-Scale Chips". In: *Proceedings of ISCA*. 2025.

[66] Saeed Rashidi, William Won, Sudarshan Srinivasan, Puneet Gupta, and Tushar Krishna. "FRED: A Wafer-Scale Fabric for 3D Parallel DNN Training". In: *Proceedings of ISCA*. 2025.

[67] Tony Chan Carusone. "Chiplet Solutions to Enable AI Scaling". In: *Optical Fiber Communication Conference (OFC)*. 2025.

[68] William J. Dally and Brian Towles. "Route Packets, Not Wires: On-Chip Interconnection Networks". In: *Proceedings of DAC*. 2001.

[69] William J. Dally and Brian Towles. *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.

[70] Peter Balfour and William J. Dally. "Design Tradeoffs for Tiled CMP On-Chip Networks". In: *Proceedings of the International Conference on Supercomputing (ICS)*. 2006.

[71] John Kim, William J. Dally, and Dennis Abts. "Flattened Butterfly: A Cost-Efficient Topology for High-Radix Networks". In: *Proceedings of ISCA*. 2007.

[72] John Kim, William J. Dally, Steve Scott, and Dennis Abts. "Technology-Driven, Highly-Scalable Dragonfly Topology". In: *Proceedings of ISCA*. 2008.

[73] Yinxiao Feng and Kaisheng Ma. "Switch-Less Dragonfly on Wafers: A Scalable Interconnection Architecture Based on Wafer-Scale Integration". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 2024.

[74] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joey Gonzalez, Hao Zhang, and Ion Stoica. "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention". In: *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. 2023.

[75] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. "Orca: A Distributed Serving System for Transformer-Based Generative Models". In: *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2022.

[76] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning". In: *Proceedings of OSDI*. 2022.

[77] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. "ServerlessLLM: Low-Latency Serverless Inference for Large Language Models". In: *Proceedings of OSDI*. 2024.

[78] Minsu Kim, Seongmin Hong, RyeoWook Ko, Soongyu Choi, Hunjong Lee, Junsoo Kim, Joo-Young Kim, and Jongse Park. "Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization". In: *Proceedings of ISCA*. 2025.

[79] Jiaming Xu, Jiayi Pan, Yongkang Zhou, Siming Chen, Jinhao Li, Yaoxiu Lian, Junyi Wu, and Guohao Dai. "SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting". In: *Proceedings of ISCA*. 2025.

[80] Shunchen Shi, Qijia Yang, Fan Yang, Yu Huang, Youwei Zhuo, Zhichun Li, Ninghui Sun, and Xueqi Li. "CoCoTree: A Computation-Capable Architecture for Collective Communication in Scalable PIM". In: *Proceedings of HPCA*. 2026.

[81] Sabuj Laskar, Pranati Majhi, Sungkeun Kim, Farabi Mahmud, Abdullah Muzahid, and E. J. Kim. "Enhancing Collective Communication in MCM Accelerator for Deep Learning Training". In: *Proceedings of HPCA*. 2024.

[82] Shu-Ting Wang, Hanyang Xu, Amin Mamandipoor, Rohan Mahapatra, Byung Hoon Ahn, Soroush Ghodrati, Krishnan Kailas, Mohammad Alian, and Hadi Esmaeilzadeh. "Data Motion Acceleration: Chaining Cross-Domain Multi Accelerators". In: *Proceedings of HPCA*. 2024.

[83] Sudhanshu Gupta and Sandhya Dwarkadas. "RELIEF: Relieving Memory Pressure in SoCs via Data Movement-Aware Accelerator Scheduling". In: *Proceedings of HPCA*. 2024.